



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación Orientada a Objetos

Práctica 6: Implementación de Asociación entre clases.

Ingeniería en Computación

Alumno:

Joshua Osorio O. – 1293271

Docente:

M.C. Itzel Barriba Cázares

Abril/2026

Tabla de contenido

Equipo y material de apoyo:	2
DESARROLLO DEL TALLER	3
Ejercicio 1.....	¡Error! Marcador no definido.
Clase Rectángulo	¡Error! Marcador no definido.
Notación UML: tipos de datos y parámetros.....	¡Error! Marcador no definido.
Notación UML: Tipos de datos y parámetros	¡Error! Marcador no definido.
Clase RectánguloDemo	¡Error! Marcador no definido.
Ejercicio 2.....	¡Error! Marcador no definido.
Diagrama UML.....	¡Error! Marcador no definido.
Clase rectángulo	¡Error! Marcador no definido.
Clase mainRectangulo	¡Error! Marcador no definido.
Resultados	¡Error! Marcador no definido.
Ejercicio 3.....	¡Error! Marcador no definido.
Clase Car	¡Error! Marcador no definido.
Implementación.....	¡Error! Marcador no definido.
Resultados	¡Error! Marcador no definido.

Equipo y material de apoyo:

- Computadora Personal (PC)
- Editor de texto Notepad
- JDK Java Development Kit

OBJETIVO

Los estudiantes implementarán asociaciones unidireccionales y bidireccionales en Java, comprendiendo cómo se establecen las relaciones entre clases.

DESARROLLO DEL TALLER

Programa 1: Sistema de pedidos.

Un sistema de comercio electrónico maneja información sobre clientes y pedidos. Un cliente puede realizar múltiples pedidos, pero cada pedido pertenece a un solo cliente.

- Identifica las clases y sus atributos esenciales
- Define los métodos necesarios para gestionar la relación entre clientes y pedidos.
- Dibuja un diagrama de clases donde represente la asociación entre Cliente y Pedido.
- Piensa en cómo se representa esta relación en código. ¿Usarás una lista de pedidos en la clase Cliente? ¿Cómo accederás a la información de un pedido?

Después de analizar el problema, implementa las clases en código y prueba su funcionalidad creando cliente y pedidos.

Cliente.java

```
public class Cliente {
    private String nombre;
    private String rfc;
    private Pedido[] pedidos; // Arreglo de objetos
    private int contadorPedidos;

    public Cliente(String nombre, String rfc, int capacidad) {
        this.nombre = nombre;
        this.rfc = rfc;
        this.pedidos = new Pedido[capacidad]; // Se define el tamaño del arreglo
        this.contadorPedidos = 0;
    }

    public void agregarPedido(Pedido p) {
        if (contadorPedidos < pedidos.length) {
            pedidos[contadorPedidos] = p;
            contadorPedidos++;
        } else {
            System.out.println("Error: Límite de pedidos alcanzado para el cliente.");
        }
    }

    public void mostrarHistorial() {
        System.out.println("\nHistorial de: " + nombre + " (RFC: " + rfc + ")");
        for (int i = 0; i < contadorPedidos; i++) {
            System.out.println(pedidos[i].getInfoPedido());
        }
    }
}
```

Pedido.java

```
public class Pedido {  
    private int idPedido;  
    private String descripcion;  
    private double monto;  
  
    public Pedido(int idPedido, String descripcion, double monto) {  
        this.idPedido = idPedido;  
        this.descripcion = descripcion;  
        this.monto = monto;  
    }  
  
    public String getInfoPedido() {  
        return String.format("ID: %-5d | Artículo: %-15s | Monto: $%.2f",  
            idPedido, descripcion, monto);  
    }  
}
```

Programa 2: Sistema universitario.

En una universidad, cada profesor está asignado a un departamento, y cada departamento tiene un único profesor a cargo.

- Análisis el problema e identifica las clases. ¿Qué atributos son esenciales para Profesor y Departamento?
- Define cómo establecerás la relación entre profesor y departamento. ¿Cada objeto tendrá una referencia al otro?
- Dibuja un diagrama de clases para visualizar la relación.
- Piensa en las operaciones necesarias. ¿Cómo se asignará un profesor a un departamento? ¿Cómo se obtendrá el departamento de un profesor?

Una vez analizado el problema, implementa la solución en código y prueba diferentes escenarios.

Departamento.java

```
public class Departamento {
    private String nombreDepto;
    private Profesor encargado;

    public Departamento(String nombreDepto) {
        this.nombreDepto = nombreDepto;
    }

    public void setEncargado(Profesor encargado) {
        this.encargado = encargado;
    }

    public String getNombreDepto() {
        return nombreDepto;
    }

    public String getInfoEncargado() {
        return (encargado != null) ? encargado.getNombre() : "Sin asignar";
    }
}
```

Profesor.java

```
public class Profesor {
    private String nombre;
    private Departamento deptoAsignado;

    public Profesor(String nombre) {
        this.nombre = nombre;
    }

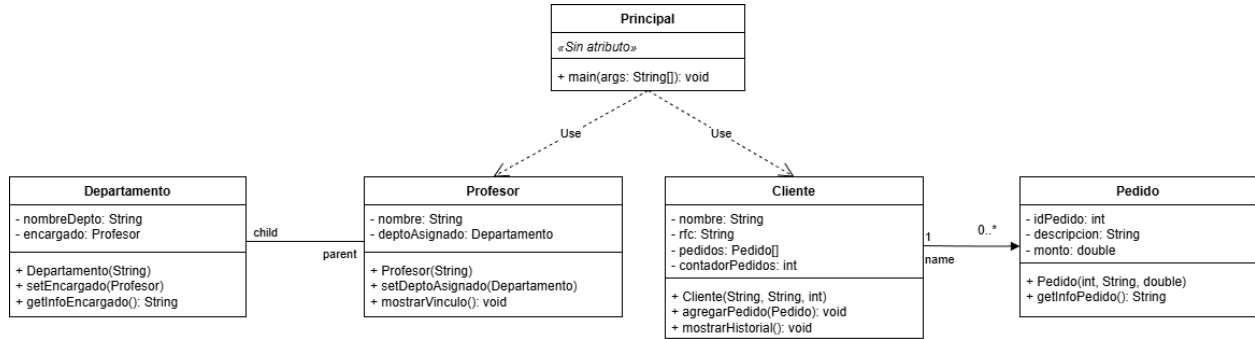
    public void setDeptoAsignado(Departamento deptoAsignado) {
        this.deptoAsignado = deptoAsignado;
    }

    public String getNombre() {
        return nombre;
    }

    public void mostrarVinculo() {
        String depto = (deptoAsignado != null) ? deptoAsignado.getNombreDepto() : "Ninguno";
        System.out.println("Profesor: " + nombre + " | Adscrito a: " + depto);
    }
}
```

Diagrama UML

UABC_FCQI_PROGRAMACION_ORIENTADA_A_OBJETOS.



1

Main.java

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("\n\t----- Menú Práctica 6 -----");
            System.out.println("\t1 - Sistema de Pedidos (1 a Muchos)");
            System.out.println("\t2 - Sistema Universitario (Bidireccional)");
            System.out.println("\t3 - Salir");
            System.out.print("\tOpción: ");
            opcion = input.nextInt();
            input.nextLine(); // Limpiar buffer

            switch (opcion) {
                case 1:
                    ejecutarProgramaPedidos(input);
                    break;
                case 2:
                    ejecutarProgramaUniversidad(input);
                    break;
                case 3:
                    System.out.println("Saliendo del programa...");
                    break;
                default:
                    System.out.println("Opción no válida.");
            }
        } while (opcion != 3);
    }
}

```

```

public static void ejecutarProgramaPedidos(Scanner input) {
    System.out.println("\n--- Registro de Cliente ---");
    System.out.print("Nombre del cliente: ");
    String nombre = input.nextLine();
    System.out.print("RFC: ");
    String rfc = input.nextLine();

    // Instancia del arreglo de objetos según tus diapositivas
    System.out.print("¿Cuántos pedidos desea registrar? ");
    int cant = input.nextInt();
    input.nextLine();

    Cliente cliente = new Cliente(nombre, rfc, cant);

    for (int i = 0; i < cant; i++) {
        System.out.println("\nDatos del Pedido " + (i + 1) + ":");
        System.out.print("ID Pedido: ");
        int id = input.nextInt();
        input.nextLine();
        System.out.print("Descripción: ");
        String desc = input.nextLine();
        System.out.print("Monto: ");
        double monto = input.nextDouble();
        input.nextLine();

        // Se crea el objeto y se asocia
        Pedido p = new Pedido(id, desc, monto);
        cliente.agregarPedido(p);
    }

    cliente.mostrarHistorial();
}

```

```

public static void ejecutarProgramaUniversidad(Scanner input) {
    System.out.println("\n--- Registro Universidad (Bidireccional) ---");
    System.out.print("Nombre del Profesor: ");
    String nomProf = input.nextLine();
    System.out.print("Nombre del Departamento: ");
    String nomDepto = input.nextLine();

    // 1. Crear instancias
    Profesor prof = new Profesor(nomProf);
    Departamento depto = new Departamento(nomDepto);

    // 2. Establecer relación bidireccional (Asociación)
    prof.setDeptoAsignado(depto);
    depto.setEncargado(prof);

    System.out.println("\n--- Verificación de Asociación ---");
    prof.mostrarVinculo();
    System.out.print("Encargado del depto " + depto.getNombreDepto() + ": " + depto.getIn-
foEncargado());
}
}

```

Resultado en terminal

```

C:\Users\Okuyt\OneDrive - uabc.edu.mx\8_Semestre\36282_ProgramacionOrientadaAObjetos\2_Laboratorio\P06_Asociacion
> cd "C:\Users\Okuyt\OneDrive - uabc.edu.mx\8_Semestre\36282_ProgramacionOrientadaAObjetos\2_Laboratorio\P06_Asociacion"
> javac -d output -sourcepath src src\*.java
> java -cp output Main

----- Menú Práctica 6 -----
1 - Sistema de Pedidos (1 a Muchos)
2 - Sistema Universitario (Bidireccional)
3 - Salir
Opción: 1

--- Registro de Cliente ---
Nombre del cliente: Joshua Osorio
RFC: 0003110602HBCSSSA6
¿Cuántos pedidos desea registrar? 2

Datos del Pedido 1:
ID Pedido: 123
Descripción: 3 Tacos adobada
Monto: 150

Datos del Pedido 2:
ID Pedido: 420
Descripción: Mula mixta
Monto: 100

Historial de: Joshua Osorio (RFC: 0003110602HBCSSSA6)
ID: 123 | Artículo: 3 Tacos adobada | Monto: $150.00
ID: 420 | Artículo: Mula mixta | Monto: $100.00

----- Menú Práctica 6 -----
1 - Sistema de Pedidos (1 a Muchos)
2 - Sistema Universitario (Bidireccional)
3 - Salir
Opción: 2

--- Registro Universidad (Bidireccional) ---
Nombre del Profesor: Itzel Barribas
Nombre del Departamento: Vinculacion

--- Verificación de Asociación ---
Profesor: Itzel Barribas | Adscrito a: Vinculacion
Encargado del depto Vinculacion: Itzel Barribas

----- Menú Práctica 6 -----
1 - Sistema de Pedidos (1 a Muchos)
2 - Sistema Universitario (Bidireccional)
3 - Salir
Opción: 3
Saliendo del programa...

```